Felicity Event Management System (Evently)

Comprehensive Design & Implementation Report

Software Systems Project

July 7, 2026

Abstract

The Felicity Event Management System (Evently) is a centralized, full-stack platform engineered using the MERN stack (MongoDB, Express.js, React.js, Node.js). It replaces traditional, fragmented event management methods—such as scattered Google Forms, manual spreadsheets, and WhatsApp groups—with a structured, role-based architecture. This comprehensive report outlines the system's architecture, data models, core implementation strategies, advanced feature workflows, project structure, and local deployment instructions. The goal of this report is to provide a complete technical overview so that any developer can immediately understand how the system was built and how it operates.

Contents

1 System Architecture & Technology Stack **3**
- 1.1 Backend Implementation . . . 3
- 1.2 Frontend Implementation . . . 3

2 Project Directory Structure **4**

3 Core System Implementation **4**
- 3.1 Authentication & Role Guards . . . 4
- 3.2 Organizer Features . . . 5
- 3.3 Participant Features . . . 5

4 Advanced Features Implementation **5**
- 4.1 Tier A: Core Advanced Features . . . 5
  - 4.1.1 Hackathon Team Registration (Feature 1) . . . 5
  - 4.1.2 QR Scanner & Attendance Tracking (Feature 3) . . . 6
- 4.2 Tier B: Real-Time & Communication Features . . . 6
  - 4.2.1 Real-Time Discussion Forum (Feature 1) . . . 6
  - 4.2.2 Organizer Password Reset Workflow (Feature 2) . . . 6
  - 4.2.3 Team Chat (Feature 3) . . . 6
- 4.3 Tier C: Integration Features . . . 7
  - 4.3.1 Anonymous Feedback System (Feature 1) . . . 7

5 Local Setup & Installation Instructions **7**
- 5.1 1. Clone the Repository . . . 7
- 5.2 2. Backend Configuration . . . 7
- 5.3 3. Frontend Configuration . . . 7

6 Conclusion **8**

1 System Architecture & Technology Stack

The system is built on a modern decoupled architecture. The backend serves as a stateless RESTful API, while the frontend operates as a highly responsive Single Page Application (SPA).

1.1 Backend Implementation

The backend relies on Node.js and Express.js to handle asynchronous I/O operations and API routing.

- **MongoDB & Mongoose (ODM):** MongoDB was chosen because its NoSQL document structure perfectly accommodates dynamic custom form schemas required for various events. Mongoose provides robust schema validation, lifecycle hooks, and relationship mapping between entities (e.g., linking Participants to Teams).
- **Authentication & Security:** We utilized JSON Web Tokens (JWT) for stateless authentication, eliminating server-side session memory overhead. Passwords are mathematically hashed using `bcryptjs` to prevent plaintext storage and brute-force attacks.
- **Real-Time Communication (Socket.io):** Integrated to power real-time features like the event discussion forums and team chats. It establishes persistent WebSocket connections with HTTP long-polling fallbacks, ensuring messages are pushed to clients with zero latency.
- **Data Validation & Utilities:** `Joi` is used for strict payload validation at the route boundary. `Nodemailer` handles automated SMTP email dispatch for QR tickets, while the `qrcode` module dynamically generates base64 QR codes on the server.

1.2 Frontend Implementation

The frontend prioritizes user experience and rapid interaction.

- **React.js & Vite:** React enables a declarative component model essential for building complex role-based dashboards. Vite was chosen over Webpack/CRA due to its extremely fast hot-module replacement (HMR) and optimized build times.
- **React Router v6:** Manages client-side routing, protected routes, and role-based access control. Unauthenticated users are securely redirected via declarative `<Navigate>` components.
- **Tailwind CSS:** Adopted as the utility-first CSS framework. It allows for rapid, custom styling directly within JSX, solving the issue of bloated global CSS files and preventing the rigid limitations often found in component libraries like Material-UI.
- **State Management & Networking:** `Axios` intercepts HTTP requests to attach JWT headers globally. `React Hook Form` handles complex dynamic registration forms with minimal re-renders.

2 Project Directory Structure

To ensure maintainability, the project is strictly divided into frontend and backend directories, utilizing an MVC-inspired pattern on the server and a component-based pattern on the client.

```
Evently/
|-- backend/
|   |-- controllers/      # Contains business logic for routes (e
   .g., event.controller.js)
|   |-- middleware/       # Express middlewares (auth.middleware.
   js, role.middleware.js)
|   |-- models/           # Mongoose schemas (User, Event, Team,
   ForumMessage)
|   |-- routes/           # Express API route definitions
|   |-- sockets/          # Socket.io event listeners and
   emitters
|   |-- utils/            # Helper functions (email sender, QR
   generator)
|   |-- server.js         # Backend entry point and database
   connection
|   |-- package.json
|
|-- frontend/
|   |-- src/
|   |   |-- components/   # Reusable UI components (Navbar,
   EventCard, ProtectedRoute)
|   |   |-- pages/        # Route-level components grouped by
   role (Admin, Organizer, Participant)
|   |   |-- context/      # React Context for global Auth and
   Socket state
|   |   |-- utils/        # Axios interceptor setup and helper
   functions
|   |   |-- App.jsx       # Main application router
|   |   |-- main.jsx      # React DOM rendering entry
|   |-- index.html
|   |-- vite.config.js
|   |-- package.json
|-- README.md
```

3 Core System Implementation

The platform enforces strict role separation. A user's role dictates their UI and their API access permissions.

3.1 Authentication & Role Guards

- **Backend Middleware:** Every protected route passes through `auth.middleware.js` which verifies the JWT, followed by `role.middleware.js` which asserts if the user's decoded role matches the route's allowed roles.

- **Frontend Guards:** The `<ProtectedRoute>` wrapper component checks the global AuthContext. If a Participant tries to access an Organizer URL, they are immediately redirected to an unauthorized page or dashboard.

3.2 Organizer Features

Organizers have a dedicated dashboard to manage the entire lifecycle of an event.

- **Event Creation:** Organizers define event metadata, limits, and deadlines. A critical feature is the **Dynamic Form Builder**, which allows organizers to define custom questions (text, dropdowns) stored as a JSON array in the MongoDB `Event` document.
- **Dashboard Analytics:** MongoDB aggregation pipelines compute live statistics (total registrations, revenue) which are displayed on the Organizer Dashboard.

3.3 Participant Features

Participants have a personalized portal.

- **Browse Events:** Implements fuzzy search and advanced filtering (by type, eligibility, date) natively on the client-side using array filtering on data fetched via Axios.
- **Registration:** When a participant registers, their responses are validated against the event's dynamic schema. Upon success, a `Registration` document is created, a unique UUID is generated, and `Nodemailer` dispatches an HTML email containing a base64 encoded QR ticket.

4 Advanced Features Implementation

To provide a highly robust application, the following advanced features were implemented across specific tiers.

4.1 Tier A: Core Advanced Features

4.1.1 Hackathon Team Registration (Feature 1)

Objective: Allow users to form teams seamlessly prior to bulk registration.
Implementation:

- **Database:** A `Team` Mongoose model tracks the `leaderId`, `requiredSize`, and an array of `members` with their pending/accepted statuses.
- **Workflow:** The leader creates the team and shares a unique invite code. Participants submit this code via the frontend. The backend updates the member array. Once the array length equals `requiredSize` and all statuses are "accepted", a Mongoose `pre-save` hook or controller function triggers a bulk generation of `Registration` documents and dispatches individual QR tickets to all members simultaneously.

4.1.2 QR Scanner & Attendance Tracking (Feature 3)

Objective: Provide a digital check-in system for physical events.
Implementation:

- **Frontend:** Utilizes the `html5-qrcode` library to turn the organizer's device camera into a scanner. The library continuously captures video frames, decodes the QR to extract the Ticket UUID, and halts scanning once decoded.
- **Backend:** The frontend sends an Axios POST request with the UUID. The controller searches the `Registration` collection. If `attendedAt` is already populated, it returns a 400 error (Duplicate Scan). If empty, it updates the timestamp and returns a success response.

4.2 Tier B: Real-Time & Communication Features

4.2.1 Real-Time Discussion Forum (Feature 1)

Objective: Facilitate live Q&A and announcements on the event page.
Implementation:

- When a user mounts the Event Detail page in React, a `Socket.io` client connects to the backend and emits a `joinRoom(eventId)` event.
- When a user submits a message, it is emitted to the server. The server saves the message to MongoDB as a `ForumMessage` document and instantly broadcasts it to all clients in the `eventId` room.

4.2.2 Organizer Password Reset Workflow (Feature 2)

Objective: Maintain administrative oversight by preventing self-serve resets for high-privilege accounts.
Implementation:

- The Organizer submits a `PasswordResetRequest` document.
- The Admin views all pending requests in their React dashboard. Upon clicking "Approve", an Axios request is sent to the backend.
- The backend uses Node's `crypto` module to generate a random 10-character string, hashes it with `bcryptjs`, saves it to the Organizer's record, and resolves the request.

4.2.3 Team Chat (Feature 3)

Objective: Provide a private, real-time collaboration space for Hackathon teams.
Implementation:

- Similar to the Discussion Forum, but the Socket namespace/room is restricted to the `teamId`. The backend validates that the requesting user's ID exists in the `Team.members` array before allowing the socket connection to join the room, ensuring absolute privacy.

4.3 Tier C: Integration Features

4.3.1 Anonymous Feedback System (Feature 1)

Objective: Collect post-event ratings without compromising privacy.
Implementation:

- Participants view a feedback modal on their dashboard for past events. Upon submission, the backend verifies their attendance via the `Registration` collection.
- The data is saved to a `Feedback` model that explicitly omits the `participantId`.
- The Organizer dashboard utilizes MongoDB's `$group` and `$avg` aggregation pipelines to compute and display the average star rating dynamically.

5 Local Setup & Installation Instructions

To run this project locally, ensure you have Node.js (v18+) and MongoDB installed (or use a MongoDB Atlas cluster).

5.1 1. Clone the Repository

```
git clone <repository_url>
cd Evently
```

5.2 2. Backend Configuration

```
cd backend
npm install
```

Create a `.env` file in the `backend` folder with the following variables:

```
PORT=5000
MONGO_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/
   evently
JWT_SECRET=your_super_secret_jwt_key
SMTP_EMAIL=your_email@gmail.com
SMTP_PASSWORD=your_gmail_app_password
```

Start the backend server:

```
npm run dev
```

5.3 3. Frontend Configuration

Open a new terminal window:

```
cd frontend
npm install
```

Create a `.env` file in the `frontend` folder:

```
VITE_API_URL=http://localhost:5000/api
VITE_SOCKET_URL=http://localhost:5000
```

Start the frontend development server:

```
npm run dev
```

The application will now be accessible at `http://localhost:5173`.

6 Conclusion

The Evently platform successfully modernizes the college fest experience by centralizing event creation, dynamic registration, and attendance tracking into a cohesive, secure platform. The integration of role-based routing, real-time WebSockets, and strict multi-actor workflows directly solves the organizational challenges of large-scale student events.